

When Answers Stop Leaking and Start Teaching: Ground-Truth-Guided Traces for Self-Improving Reasoners

Anonymous ACL submission

Abstract

High-quality reasoning traces are a key ingredient for improving reasoning models, yet producing them often requires stronger teacher models or human-written rationales. We show that ground-truth answers can instead serve as privileged guidance for synthesizing high-value, model-native training traces. Our pipeline conditions a reasoner on both a problem and its known answer, allowing ordinary question-answer data to yield substantially stronger solution trajectories. This advantage introduces *privileged leakage*: generated traces may explicitly mention the supplied answer or rely on it as evidence. To make these traces valid beyond the privileged generation setting, we calibrate a sparse attention-based detector that identifies tokens associated with leakage. When the detector fires during generation, the decoder rolls back the affected region and rewrites it using a synchronized clean branch that never observes the answer. The resulting ground-truth-guided traces preserve the central benefits of privileged generation: the known outcome steers generation toward correct solutions and supports self-verification throughout the reasoning process. Meanwhile, clean-branch repair removes explicit reliance on privileged context while preserving fluent and natural reasoning trajectories. Across mathematical, scientific, logical, and multi-hop reasoning domains, fine-tuning on only 1K–2K generated samples consistently improves reasoning performance. On the MATH domain, the resulting data improves DeepSeek-R1-Distill-Qwen-14B by up to 14 points. These results establish ground-truth-guided trace synthesis as an effective source of reasoning supervision for bootstrapping self-improving reasoners without external rationale supervision.

1 Introduction

Chain-of-thought (CoT) supervision can transfer reasoning behavior to smaller language models

and improve performance on multi-step tasks (Wei et al., 2022; Ho et al., 2023). However, creating such supervision at scale remains difficult. Unconditioned sampling often produces incorrect answers, while filtering for correctness discards potentially useful examples and spends compute on failed trajectories. A natural alternative is to expose the known answer while asking the generator to construct a derivation. This form of answer-conditioned rationalization is used explicitly in STaR (Zelikman et al., 2022) and is closely related to contrastive answer-conditioned decoding in SCOTT (Wang et al., 2023a).

Answer conditioning creates a train-test asymmetry. The generator may mention that the answer was supplied, verify its work against the reference, or treat the target as a premise. We call these behaviors *privileged leakage*. They are distinct from legitimately deriving and stating the same final answer: the issue is not the appearance of the answer value itself, but reliance on information unavailable to the eventual student. A trace such as “since the given answer is 72” is therefore invalid supervision even when its arithmetic is correct. Lexical filters can catch fixed phrases, but leakage can be paraphrased, distributed across several tokens, or expressed without words such as *answer* or *ground truth*.

This paper asks whether the generator’s own attention can reveal when privileged information is about to surface. Our starting observation is that only a small number of attention heads need to align strongly with human- or judge-annotated leakage spans. This agrees with broader evidence that sparse attention heads can expose task-relevant model behavior (Hung et al., 2024). We use the mass assigned by these heads to the privileged prompt span as a token-level signal. Importantly, detection is only half of the problem: discarding the full trajectory wastes its valid prefix, while merely suppressing one token can defer the

same leakage to the next step.

We propose TRACEGUARD, a detect–rollback–regenerate procedure for constructing reasoning supervision. Generation maintains two views of the same output prefix: a *privileged branch* conditioned on the question and answer, and a *clean branch* conditioned on the question alone. The privileged branch normally proposes tokens. If a proposed token attends too strongly to the answer span, the decoder backtracks to the beginning of the suspected region and uses the clean branch to rewrite it. Both branches then consume the chosen tokens so that they remain synchronized. Repair ends only after the detector observes a configurable safe streak, preventing immediate oscillation back into the same failure mode.

Our contributions are:

- We formulate *privileged leakage* as a distinct data-quality problem in answer-conditioned reasoning-trace generation.
- We develop a lightweight token-level detector by calibrating a sparse set of attention heads against annotated leakage spans.
- We introduce online local repair that combines bounded backtracking, a clean counterfactual branch, synchronized decoding, and KV-cache reuse.
- We provide preliminary detector validation across four Qwen3 model scales and an evaluation plan for trace quality, downstream utility, and computational overhead.

2 Related Work

Reasoning data generation and distillation.

CoT prompting elicits intermediate reasoning from sufficiently capable models (Wei et al., 2022), while self-consistency improves answer selection by sampling multiple reasoning paths (Wang et al., 2023b). Fine-tune-CoT uses teacher-generated rationales as supervision (Ho et al., 2023), and STaR bootstraps rationales by retrying failed questions with the correct answer supplied (Zelikman et al., 2022). SCOTT conditions rationale generation on the target answer through contrastive decoding and emphasizes consistency between rationales and predictions (Wang et al., 2023a). These approaches motivate using target information during data construction; our focus is the complementary problem of ensuring that the

resulting text does not reveal this privileged construction process.

Faithfulness and internal signals. Free-text rationales need not faithfully describe the computation supporting a prediction, and their surface plausibility alone is therefore insufficient (Jacovi and Goldberg, 2020). Work on attention-based prompt-injection detection shows that selected heads can provide useful inference-time signals without an auxiliary model call (Hung et al., 2024). We similarly calibrate sparse heads, but target a different phenomenon: token-level transfer from a known answer in the prompt into a generated rationale. We additionally couple detection to a causal intervention—regeneration without the privileged context—rather than using the signal only for classification.

3 Method

3.1 Problem Formulation

Let a supervised reasoning dataset contain pairs $(\mathbf{x}, \mathbf{y})$, where $\mathbf{x}$ is a problem and $\mathbf{y}$ is its known answer. We wish to produce a reasoning trace $\mathbf{z} = (z_1, \dots, z_T)$ that is useful for supervised fine-tuning under the clean input $\mathbf{x}$. During data construction, however, we allow the generator p_θ to observe a privileged context $c(\mathbf{y})$, such as “Given the ground-truth answer is $\mathbf{y}$.”:

$$z_t \sim p_\theta(\cdot \mid \mathbf{x}, c(\mathbf{y}), z_{<t}). \quad (1)$$

The desired trace may derive and state $\mathbf{y}$, but it should not explicitly refer to $c(\mathbf{y})$, cite the supplied target as evidence, or express awareness that privileged information was provided. We represent these undesirable positions with a binary token mask $g \in \{0, 1\}^T$ and learn a detector D_t that estimates whether generation around position t depends on the privileged span in a way associated with leakage.

3.2 Constructing Leakage Annotations

We first sample responses from the generator with $c(\mathbf{y})$ appended to the problem. A separate judge receives the privileged prompt and response and returns exact text spans exhibiting one of four behaviors: (i) directly referencing the provided answer, (ii) checking a result against it, (iii) using it as a reasoning constraint, or (iv) discussing the need to hide it. Merely deriving or stating the final value is labeled clean. Character spans are mapped

to response-token spans through tokenizer offsets. We split annotated examples into head-calibration, validation, and held-out test sets.

Because judge-generated span boundaries may be imperfect, our primary detector metric is *span-start recall*: a gold span is detected when its first token is flagged. We additionally report span precision, token precision, and *full recall*, the fraction of examples in which every annotated span is detected. **[TODO: Report annotation protocol, judge model, number of examples, inter-annotator agreement or a human audit, and the exact data split.]**

3.3 Sparse Attention-Based Leakage Detector

Let $A_{t,j}^{\ell,h}$ be the causal attention weight from response token t to prompt token j at layer ℓ and head h . If I_c is the set of token indices occupied by $c(\mathbf{y})$, the privileged attention mass is

$$a_t^{\ell,h} = \sum_{j \in I_c} A_{t,j}^{\ell,h}. \quad (2)$$

We score each head on the calibration split in two stages. First, we remove its within-example background level using a positive median normalization,

$$\tilde{a}_t^{\ell,h} = \max(0, a_t^{\ell,h} - \text{median}_u a_u^{\ell,h}). \quad (3)$$

Second, we reward both alignment with the gold leakage mask and separation between positive and negative tokens:

$$q_{\ell,h} = \cos(\tilde{\mathbf{a}}^{\ell,h}, \mathbf{g}) \cdot \left[\text{mean}_{t:g_t=1} a_t^{\ell,h} - \text{mean}_{t:g_t=0} a_t^{\ell,h} \right]_+. \quad (4)$$

Scores are averaged over calibration examples, and the top K heads are kept. Their normalized signals are combined using non-negative calibration scores:

$$D_t = \frac{\sum_{(\ell,h) \in \mathcal{H}} q_{\ell,h} \tilde{a}_t^{\ell,h}}{\sum_{(\ell,h) \in \mathcal{H}} q_{\ell,h}}. \quad (5)$$

We apply a centered rolling maximum of width w to tolerate boundary noise and flag positions whose score exceeds τ . The tuple (K, w, τ) is selected on the validation split. Our default selection favors configurations with high example-level full recall, then balances span recall and precision

while preferring fewer heads and shorter windows. This produces a model-specific detector with no learned classifier parameters.

Online score. At decode time, only the selected detector layers return attention matrices; other layers may use a fused attention backend. This avoids materializing full-model attention for every token. The current implementation uses the calibrated heads’ weighted privileged attention mass for the candidate token and uses w to determine the repair radius.

3.4 Dual-Branch Online Repair

For each example, we maintain two autoregressive branches sharing model parameters but using different prompt caches:

$$p_{\text{priv}}(z_t) = p_{\theta}(z_t \mid \mathbf{x}, c(\mathbf{y}), z_{<t}), \quad (6)$$

$$p_{\text{clean}}(z_t) = p_{\theta}(z_t \mid \mathbf{x}, z_{<t}). \quad (7)$$

Both branches always consume the accepted output token, so they condition on an identical generated prefix even though their prompt contexts differ. Normal decoding samples a candidate $\hat{z}_t$ from p_{priv} and computes its detector score. If the score is below τ , the token is accepted on both branches. Otherwise, the method initiates repair.

Let $r = \lfloor (w - 1)/2 \rfloor$. Upon detecting leakage at position t , we roll the privileged branch back to $s = \max(1, t - r)$ and discard $z_{s:t}$ together with the rejected candidate. The clean branch is restored to the same accepted prefix. Tokens covering the backtracked region are sampled from p_{clean} and consumed by both branches. Thereafter, we continue sampling clean replacements while a privileged candidate remains unsafe. To avoid ending repair at an isolated low-score token, generation returns to normal only after r consecutive privileged candidates pass the detector. An alternative comparison uses the Jensen–Shannon divergence between privileged and clean next-token distributions:

$$m = \frac{1}{2}(p_{\text{priv}} + p_{\text{clean}}), \quad (8)$$

$$\text{JSD}(p_{\text{priv}}, p_{\text{clean}}) = \frac{1}{2} \text{KL}(p_{\text{priv}} \parallel m) + \frac{1}{2} \text{KL}(p_{\text{clean}} \parallel m). \quad (9)$$

This alternative is useful in ablations but doubles next-token distribution comparisons and lacks the sparse attribution supplied by the attention detector.

Termination and anti-oscillation. Repair is capped at a fixed number of steps. If repeated detections would backtrack to the same prefix more than a configured number of cycles, we suppress further rollback at that location and replace only the newly proposed token. These guards make the procedure terminate or fail explicitly instead of silently looping. All repair events—detection score, removed tokens, replacement tokens, and branch choices—are logged for audit.

3.5 Efficient Decoding

Naively restarting after every repair would repeatedly prefill the full prompt and accepted trace. We instead crop each branch’s dynamic KV cache to the common prefix and recompute only the missing suffix. During ordinary decoding, samples with different cache lengths are padded and batched; after a forward pass, caches are unpadded and returned to per-sample state. On multi-GPU nodes, we run one model replica per GPU and distribute examples dynamically. These optimizations do not change the detector or token-selection rules, although different fused kernels and batch shapes may introduce small numerical changes near a sampling or detection boundary.

3.6 Exporting Traces for Supervised Fine-Tuning

The final training example contains the clean user message $\mathbf{x}$ and the repaired assistant trace $\mathbf{z}$; $c(\mathbf{y})$ is never exported. We retain the answer value when it is naturally derived or stated. Truncated generations may optionally be excluded, and generation metadata and repair logs are stored separately from the SFT file. Thus, the privileged answer acts only as a data-construction aid, not as student input.

4 Preliminary Detector Evaluation

Table 1 reports the detector metrics currently stored by the repository for four Qwen3 scales (Yang et al., 2025). Results are from model-specific detectors and held-out leakage-annotated responses. These operating points intentionally favor recall because a missed span is exported into training data, whereas a false positive triggers a local clean rewrite. The high recall therefore comes with modest span precision, especially for 14B and 32B.

Model	Heads	Span P	Span R	Full R
Qwen3-4B	1	21.5	100.0	100.0
Qwen3-8B	1	38.5	99.3	98.2
Qwen3-14B	4	8.2	99.2	98.0
Qwen3-32B	1	13.4	99.6	98.4

Table 1: Preliminary held-out leakage detection results (%). “Full R” is the percentage of examples for which all annotated leakage spans are detected. These values validate the detector only; they do not yet establish that online repair improves final traces or downstream models.

5 Experimental Plan

[TODO: Replace this section with completed experiments.] We propose evaluating the full pipeline on mathematical, scientific, logical, and multi-hop reasoning benchmarks. For each base generator, compare:

1. clean generation without the answer;
2. privileged generation without filtering;
3. privileged generation with lexical/rule-based filtering;
4. rejection sampling that discards any detected trace;
5. TRACEGUARD with attention-based local repair; and
6. TRACEGUARD with JSD-based repair.

The primary trace-level outcomes should be answer accuracy, leakage rate under a blinded human audit and an independent judge, retention rate, trace length, and generation tokens or wall-clock time per accepted example. To test usefulness rather than surface cleanliness alone, fine-tune identical student models on equal-sized trace sets and report clean-input benchmark accuracy. Detector ablations should vary the number of heads, calibration size, window length, threshold, backtracking, safe-streak rule, and clean-branch synchronization. Cross-model and cross-domain transfer will reveal whether calibration must be repeated for each generator and task family.

6 Limitations

The current evidence validates a detector on automatically annotated leakage spans but does not

yet demonstrate improved downstream reasoning. The judge may miss subtle semantic dependence on the supplied answer, and attention is not a complete causal explanation of generation. Model-specific heads and thresholds may not transfer across architectures, checkpoints, domains, prompt templates, or precision settings. Recall-oriented thresholds can cause unnecessary clean rewrites, reducing the advantage of privileged conditioning. Maintaining two prompt caches and selectively materializing attention increases memory and latency relative to ordinary decoding. Finally, a repaired trace can remain factually incorrect or unfaithful even when it contains no explicit privileged leakage. We therefore treat leakage repair as one data-quality filter, not a guarantee of correct or faithful reasoning.

7 Ethical Considerations

The method is intended to reduce misleading supervision in synthetic reasoning data. Nevertheless, automatically generated rationales can convey false confidence, stereotypes, or unsafe content unrelated to answer leakage. Releasing generated traces should include their provenance, generator version, filter settings, and known failure modes. Any human annotation study should use appropriate consent, compensation, and data-handling procedures. **[TODO: Add the dataset licenses, compute budget, annotator details, and responsible-NLP checklist responses used in the final study.]**

8 Conclusion

We presented TRACEGUARD, a draft framework for generating answer-conditioned reasoning traces without exporting explicit reliance on the answer. A sparse attention detector identifies likely leakage online, and a synchronized clean branch locally rewrites the affected region while preserving the valid prefix. Preliminary experiments support the feasibility of high-recall detection across several model scales. The decisive next step is an end-to-end evaluation of whether repaired traces preserve the accuracy benefit of privileged generation while improving data cleanliness, student performance, and generation efficiency.

References

Namgyu Ho, Laura Schmid, and Se-Young Yun. 2023. *Large language models are reasoning teachers*. In

Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 14852–14882, Toronto, Canada. Association for Computational Linguistics.

Kuo-Han Hung, Ching-Yun Ko, Ambrish Rawat, I-Hsin Chung, Winston H. Hsu, and Pin-Yu Chen. 2024. *Attention tracker: Detecting prompt injection attacks in LLMs*. *arXiv preprint arXiv:2411.00348*.

Alon Jacovi and Yoav Goldberg. 2020. *Towards faithfully interpretable NLP systems: How should we define and evaluate faithfulness?* In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4198–4205, Online. Association for Computational Linguistics.

Peifeng Wang, Zhengyang Wang, Zheng Li, Yifan Gao, Bing Yin, and Xiang Ren. 2023a. *SCOTT: Self-consistent chain-of-thought distillation*. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5546–5558, Toronto, Canada. Association for Computational Linguistics.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023b. *Self-consistency improves chain of thought reasoning in language models*. In *International Conference on Learning Representations*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. 2022. *Chain-of-thought prompting elicits reasoning in large language models*. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, et al. 2025. *Qwen3 technical report*. *arXiv preprint arXiv:2505.09388*.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. 2022. *STaR: Bootstrapping reasoning with reasoning*. In *Advances in Neural Information Processing Systems*, volume 35, pages 15476–15488.

A Draft Reproducibility Checklist

The final version should report: model identifiers and revisions; tokenizer and chat template; attention backend; numerical precision; generation temperature, top-*p*, token budget, and random seeds; the exact privileged-context template; detector heads, weights, threshold, and window for every model; repair limits; annotation prompts; train/validation/test splits; hardware; runtime; and all filtering decisions. The implementation already records per-example seeds, token IDs, repair events, detector-config hashes, and resumable run metadata.